Creacion de pedidos de una linea y multilinea.

Script inicial:

```
CREATE SCHEMA tienda;
SET search_path TO tienda;
CREATE TABLE clientes (
cliente_id SERIAL PRIMARY KEY,
nombre TEXT NOT NULL,
email TEXT UNIQUE NOT NULL,
fecha_alta DATE NOT NULL DEFAULT CURRENT_DATE
);
CREATE TABLE productos (
producto_id SERIAL PRIMARY KEY,
nombre TEXT NOT NULL,
precio NUMERIC(10,2) NOT NULL,
stock INTEGER NOT NULL DEFAULT 0,
activo BOOLEAN NOT NULL DEFAULT TRUE
);
CREATE TABLE pedidos (
pedido_id SERIAL PRIMARY KEY,
cliente_id INTEGER NOT NULL REFERENCES clientes(cliente_id),
fecha_pedido TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
estado TEXT NOT NULL DEFAULT 'pendiente'
);
CREATE TABLE lineas_pedido (
linea_id SERIAL PRIMARY KEY,
pedido_id INTEGER NOT NULL REFERENCES pedidos(pedido_id),
producto_id INTEGER NOT NULL REFERENCES productos(producto_id),
cantidad INTEGER NOT NULL,
precio_unit NUMERIC(10,2) NOT NULL
);
CREATE TABLE pagos (
pago_id SERIAL PRIMARY KEY,
pedido_id INTEGER NOT NULL REFERENCES pedidos(pedido_id),
fecha_pago TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
importe NUMERIC(10,2) NOT NULL,
metodo TEXT NOT NULL
);
CREATE TABLE empleados (
id SERIAL PRIMARY KEY,
nombre TEXT NOT NULL,
salario NUMERIC(10,2) NOT NULL,
departamento TEXT,
fecha_contratacion DATE DEFAULT CURRENT_DATE
);
INSERT INTO empleados (nombre, salario, departamento) VALUES
('Ana Torres', 2800.00, 'Ventas'),
('Luis Martínez', 4500.00, 'IT'),
('Carla Gómez', 5200.00, 'IT'),
('Dmitry Komarov', 3100.00, 'Contabilidad'),
('James Smith', 7000.00, 'Dirección'),
('Mary Sexton', 3900.00, 'Marketing'),
('Agnes Miller', 6100.00, 'IT'),
('Chun Yen', 2950.00, 'Ventas');
INSERT INTO clientes (nombre, email) VALUES
('Juan Pérez', 'juan@example.com'),
('María López', 'maria@example.com');
INSERT INTO productos (nombre, precio, stock) VALUES
```

('Teclado mecánico', 79.90, 50),
('Ratón inalámbrico', 39.50, 100),
('Monitor 27"', 249.00, 20);
INSERT INTO pedidos (cliente_id, estado) VALUES
(1, 'pendiente'),
(2, 'pendiente');
INSERT INTO lineas_pedido (pedido_id, producto_id, cantidad, precio_unit) VALUES
(1, 1, 1, 79.90),
(1, 2, 2, 39.50),
(2, 3, 1, 249.00);
INSERT INTO pagos (pedido_id, importe, metodo) VALUES
(1, 158.90, 'tarjeta'),
(2, 249.00, 'transferencia');

Procedure para Crear un pedido, crear su línea y reducir stock de manera atómica.

```
CREATE OR REPLACE PROCEDURE tienda.sp_crear_linea_pedido(
    IN p_cliente_id integer,
    IN p_producto_id integer,
    IN p_cantidad integer,
    OUT v_pedido_id integer,
    OUT v_linea_id integer,
    OUT v_total_linea numeric(10,2)
)
LANGUAGE plpgsql
AS $$
DECLARE
    v_precio_unit numeric(10,2);
    v_stock integer;
BEGIN
    IF p_cantidad <= 0 THEN
        RAISE EXCEPTION 'La cantidad debe ser mayor que cero';
    END IF;

    -- FOR UPDATE evita que dos ventas concurrentes descuenten el mismo stock.
    SELECT precio, stock
      INTO STRICT v_precio_unit, v_stock
      FROM tienda.productos
     WHERE producto_id = p_producto_id
       AND activo
     FOR UPDATE;

    IF v_stock < p_cantidad THEN
        RAISE EXCEPTION 'Stock insuficiente para producto %: disponible %,
solicitado %',
            p_producto_id, v_stock, p_cantidad;
    END IF;

    INSERT INTO tienda.pedidos (cliente_id, estado)
    VALUES (p_cliente_id, 'pendiente')
    RETURNING pedido_id INTO v_pedido_id;

    INSERT INTO tienda.lineas_pedido (pedido_id, producto_id, cantidad,
precio_unit)
    VALUES (v_pedido_id, p_producto_id, p_cantidad, v_precio_unit)
    RETURNING linea_id INTO v_linea_id;
```

```
    UPDATE tienda.productos
       SET stock = stock - p_cantidad
     WHERE producto_id = p_producto_id;

    v_total_linea := round(p_cantidad * v_precio_unit, 2);
EXCEPTION
    WHEN NO_DATA_FOUND THEN
        RAISE EXCEPTION 'Producto activo no encontrado: %', p_producto_id;
END;
$$;


-- Los lugares para OUT se pasan como NULL en CALL.
CALL tienda.sp_crear_linea_pedido(1, 1, 2, NULL, NULL, NULL);
```

Resultado esperado de `CALL`: una fila de salida con un nuevo `v_pedido_id`, el nuevo `v_linea_id` y `v_total_linea = 159.80`. Además, el stock del producto 1 disminuye en 2.

procedimiento convertido a función

Aquí tienes la versión como `FUNCTION`. La lógica interna es **idéntica**; solo cambia la firma y cómo se declaran los valores de retorno (en vez de `OUT`, se usa `RETURNS TABLE`).

sql

```
CREATE OR REPLACE FUNCTION tienda.fn_crear_linea_pedido(
    p_cliente_id  integer,
    p_producto_id integer,
    p_cantidad    integer
)
RETURNS TABLE(
    v_pedido_id   integer,
    v_linea_id    integer,
    v_total_linea numeric(10,2)
)
LANGUAGE plpgsql
AS $$
DECLARE
    v_precio_unit numeric(10,2);
    v_stock       integer;
BEGIN
    IF p_cantidad <= 0 THEN
        RAISE EXCEPTION 'La cantidad debe ser mayor que cero';
    END IF;

    -- FOR UPDATE evita que dos ventas concurrentes descuenten el mismo stock.
    SELECT precio, stock
      INTO STRICT v_precio_unit, v_stock
      FROM tienda.productos
     WHERE producto_id = p_producto_id
       AND activo
     FOR UPDATE;

    IF v_stock < p_cantidad THEN
        RAISE EXCEPTION 'Stock insuficiente para producto %: disponible %,
solicitado %',
            p_producto_id, v_stock, p_cantidad;
    END IF;
```

```
    INSERT INTO tienda.pedidos (cliente_id, estado)
    VALUES (p_cliente_id, 'pendiente')
    RETURNING pedido_id INTO v_pedido_id;

    INSERT INTO tienda.lineas_pedido (pedido_id, producto_id, cantidad,
precio_unit)
    VALUES (v_pedido_id, p_producto_id, p_cantidad, v_precio_unit)
    RETURNING linea_id INTO v_linea_id;

    UPDATE tienda.productos
       SET stock = stock - p_cantidad
     WHERE producto_id = p_producto_id;

    v_total_linea := round(p_cantidad * v_precio_unit, 2);

    RETURN NEXT;   -- emite la fila con los valores actuales de las variables de
salida
    RETURN;
EXCEPTION
    WHEN NO_DATA_FOUND THEN
        RAISE EXCEPTION 'Producto activo no encontrado: %', p_producto_id;
END;
$$;
```

Qué cambió respecto al procedimiento

Aspecto	Procedimiento original	Función nueva
Cabecera	`PROCEDURE ... (IN ..., OUT ...)`	`FUNCTION ... (IN ...) RETURNS TABLE(...)`
Parámetros de salida	`OUT v_pedido_id`, `OUT v_linea_id`, `OUT v_total_linea`	Columnas de `RETURNS TABLE(...)`
Variables de salida	Declaradas automáticamente por los `OUT`	Declaradas automáticamente por `RETURNS TABLE` (mismos nombres)
Emisión del resultado	Implícita al terminar el procedimiento	Explícita con `RETURN NEXT` + `RETURN`
Invocación	`CALL ...`	`SELECT * FROM ...`
Uso en `SELECT`/`JOIN`/`WHERE`	No	Sí

Detalles importantes

1. Los nombres de las columnas de `RETURNS TABLE` son variables locales

En PL/pgSQL, las columnas declaradas en `RETURNS TABLE` **se comportan como variables** dentro de la función. Por eso puedes hacer:

sql

```
RETURNING pedido_id INTO v_pedido_id;
```

sin declarar `v_pedido_id` en el bloque `DECLARE`: ya existe como variable de salida. Lo mismo con `v_linea_id` y `v_total_linea`.

Cuidado con la ambigüedad de nombres: si dentro de la función haces un `SELECT` que mencione una columna llamada igual que una variable de `RETURNS TABLE` (por ejemplo, si `tienda.pedidos` tuviera una columna `v_pedido_id`), PostgreSQL podría confundirse. En tu caso no ocurre porque los nombres `v_*` no coinciden con columnas reales. Si alguna vez coinciden, califica la columna con el alias de la tabla (`p.pedido_id`) o renombra la variable.

2. `RETURN NEXT` vs `RETURN QUERY`

•**`RETURN NEXT;`** (sin argumentos) emite **una fila** con los valores actuales de las variables de salida. Es lo que usamos aquí: una sola fila.

•**`RETURN QUERY ...`** emite **todas las filas** de una consulta. No es lo que queremos en este caso, porque la función hace inserciones y devuelve una sola fila de resultado.

3. `RETURN;` final

Después del `RETURN NEXT`, el `RETURN;` sin valor marca el fin de la función. En la práctica, al llegar al `END` también terminaría, pero ponerlo explícito es más claro y evita que el `EXCEPTION` se dispare por accidente si añades código después.

4. El bloque `EXCEPTION` sigue funcionando igual

`NO_DATA_FOUND` lo lanza `INTO STRICT` cuando el `SELECT ... FOR UPDATE` no encuentra fila (producto inexistente o inactivo). Igual que en el procedimiento.

5. `STABLE` no aplica aquí

No pongas `STABLE` (ni `IMMUTABLE`) en esta función: **modifica datos** (`INSERT`, `UPDATE`). Debe ser `VOLATILE`, que es el valor por defecto. Si la marcas `STABLE`, PostgreSQL podría cachear el resultado o rechazar la escritura.

Cómo llamarla

sql

```
-- Devuelve una fila con las tres columnas
SELECT * FROM tienda.fn_crear_linea_pedido(1, 1, 2);
```

Resultado:

text

```
 v_pedido_id | v_linea_id | v_total_linea
-------------+------------+---------------
           1 |          1 |         25.00
```

Y ahora **sí** puedes usarla en contextos donde un procedimiento no se puede, por ejemplo:

sql

```
-- Insertar el resultado en otra tabla
INSERT INTO tienda.auditoria (pedido_id, linea_id, total)
SELECT * FROM tienda.fn_crear_linea_pedido(1, 1, 2);

-- Usarla en un JOIN (poco común pero posible)
SELECT p.cliente_id, f.v_total_linea
  FROM tienda.clientes p
  CROSS JOIN LATERAL tienda.fn_crear_linea_pedido(1, 1, 2) f;
```

Nota sobre transacciones

Al ser `FUNCTION`, **no puede hacer `COMMIT`/`ROLLBACK`** internamente. Corre dentro de la transacción del llamador: si el `SELECT`/`INSERT` que la invoca falla después, todo se revierte junto. Para este caso (crear pedido + línea + descontar stock de forma atómica) eso es exactamente lo que quieres. Si algún día necesitas confirmar por etapas dentro de la rutina, tendrías que volver al `PROCEDURE`.

Procedimiento para crear un pedido con varias líneas

La idea: recibir el cliente y un **array de líneas** (producto + cantidad), y procesarlas todas dentro del **mismo pedido** y la **misma transacción**. Si una línea falla (stock insuficiente, producto inactivo, cantidad inválida), **todo el pedido se revierte** gracias al bloque `EXCEPTION`.

Diseño de los parámetros

Para pasar varias líneas uso un **array de un tipo compuesto** o, más simple y portable, un **array de `integer`** con cantidades en posiciones pares... pero eso es feo. Mejor: dos arrays paralelos (`p_producto_ids`, `p_cantidades`) o un tipo compuesto. Voy con **dos arrays paralelos** porque es lo más simple de llamar desde SQL y desde PL/pgSQL sin crear tipos.

Alternativa más limpia si puedes crear tipos: `CREATE TYPE tienda.linea_input AS (producto_id integer, cantidad integer);` y recibir `p_lineas tienda.linea_input[]`. Lo dejo comentado al final.

El procedimiento

sql

```
CREATE OR REPLACE PROCEDURE tienda.sp_crear_pedido_multilinea(
    IN  p_cliente_id    integer,
    IN  p_producto_ids  integer[],
    IN  p_cantidades    integer[],
    OUT v_pedido_id     integer,
    OUT v_total_pedido  numeric(12,2),
    OUT v_num_lineas    integer
)
```

```
LANGUAGE plpgsql
AS $$
DECLARE
    v_precio_unit numeric(10,2);
    v_stock       integer;
    v_linea_id    integer;
    v_total_linea numeric(10,2);
    v_i           integer;
BEGIN
    -- 1) Validaciones de entrada
    IF p_producto_ids IS NULL OR p_cantidades IS NULL
       OR array_length(p_producto_ids, 1) IS NULL
       OR array_length(p_cantidades, 1) IS NULL THEN
        RAISE EXCEPTION 'Debe indicar al menos una línea (producto y cantidad)';
    END IF;

    IF array_length(p_producto_ids, 1) <> array_length(p_cantidades, 1) THEN
        RAISE EXCEPTION 'Los arrays de productos (%) y cantidades (%) deben
tener la misma longitud',
            array_length(p_producto_ids, 1), array_length(p_cantidades, 1);
    END IF;

    -- 2) Crear la cabecera del pedido UNA sola vez
    INSERT INTO tienda.pedidos (cliente_id, estado)
    VALUES (p_cliente_id, 'pendiente')
    RETURNING pedido_id INTO v_pedido_id;

    v_total_pedido := 0;
    v_num_lineas   := 0;

    -- 3) Recorrer las líneas
    FOR v_i IN 1 .. array_length(p_producto_ids, 1) LOOP
        IF p_cantidades[v_i] IS NULL OR p_cantidades[v_i] <= 0 THEN
            RAISE EXCEPTION 'Línea %: la cantidad debe ser mayor que cero
(recibido %)',
                v_i, p_cantidades[v_i];
        END IF;

        -- Bloqueo de la fila del producto para evitar sobreventa concurrente
        SELECT precio, stock
          INTO STRICT v_precio_unit, v_stock
          FROM tienda.productos
         WHERE producto_id = p_producto_ids[v_i]
           AND activo
         FOR UPDATE;

        IF v_stock < p_cantidades[v_i] THEN
            RAISE EXCEPTION 'Línea %: stock insuficiente para producto %
(disponible %, solicitado %)',
                v_i, p_producto_ids[v_i], v_stock, p_cantidades[v_i];
        END IF;

        INSERT INTO tienda.lineas_pedido (pedido_id, producto_id, cantidad,
precio_unit)
        VALUES (v_pedido_id, p_producto_ids[v_i], p_cantidades[v_i],
v_precio_unit)
        RETURNING linea_id INTO v_linea_id;

        UPDATE tienda.productos
           SET stock = stock - p_cantidades[v_i]
         WHERE producto_id = p_producto_ids[v_i];
```

```
        v_total_linea  := round(p_cantidades[v_i] * v_precio_unit, 2);
        v_total_pedido := v_total_pedido + v_total_linea;
        v_num_lineas   := v_num_lineas + 1;
    END LOOP;

EXCEPTION
    WHEN NO_DATA_FOUND THEN
        RAISE EXCEPTION 'Producto activo no encontrado en la línea %: id %',
            v_i, p_producto_ids[v_i];
END;
$$;
```

Cómo llamarlo

Desde SQL puro, con `CALL` y `NULL` en los `OUT`:

sql

```
CALL tienda.sp_crear_pedido_multilinea(
    1,                          -- cliente
    ARRAY[1, 3, 2],             -- productos
    ARRAY[2, 1, 4],             -- cantidades
    NULL, NULL, NULL            -- OUT
);
```

Devuelve una fila con `v_pedido_id`, `v_total_pedido`, `v_num_lineas`.

Desde otro bloque PL/pgSQL, con variables para recibir los `OUT`:

sql

```
DO $$
DECLARE
    r_pedido_id integer;
    r_total     numeric(12,2);
    r_lineas    integer;
BEGIN
    CALL tienda.sp_crear_pedido_multilinea(
        1, ARRAY[1,3,2], ARRAY[2,1,4],
        r_pedido_id, r_total, r_lineas
    );
    RAISE NOTICE 'Pedido % creado con % líneas, total %',
        r_pedido_id, r_lineas, r_total;
END;
$$;
```

Puntos clave del diseño

1. Un solo `INSERT` en `pedidos`

La cabecera se crea **antes** del bucle, una sola vez. Todas las líneas comparten el mismo `v_pedido_id`. Eso es lo que pedías: "varias líneas en un mismo pedido".

2. Todo o nada

El bloque `EXCEPTION` captura cualquier fallo (stock insuficiente, producto inactivo, cantidad inválida). Al lanzar `RAISE EXCEPTION`, **toda la transacción del procedimiento se revierte**,

incluido el `INSERT` de `pedidos` y los `UPDATE` de stock ya hechos en líneas anteriores. No queda pedido huérfano ni stock descontado a medias.

Esto funciona así porque el procedimiento **no hace `COMMIT` intermedio**. Si lo hicieras, perderías la atomicidad. Por eso aquí **no** se usa `COMMIT` dentro del bucle.

3. `FOR UPDATE` dentro del bucle

Bloquea cada producto **en el momento de procesar su línea**. Si dos sesiones intentan vender el mismo producto a la vez, la segunda espera a que la primera libere el lock (al terminar su transacción) y entonces ve el stock ya descontado. Evita sobreventa.

Orden de bloqueo: si dos pedidos incluyen los mismos productos en **orden distinto** (`[1,3]` vs `[3,1]`), puede haber **deadlock**. Si te preocupa, ordena los arrays por `producto_id` antes de recorrerlos. Para un sistema pequeño no suele ser problema, pero conviene saberlo.

4. Validación de longitudes

`array_length(..., 1)` devuelve `NULL` para arrays vacíos, por eso se comprueba con `IS NULL`. Y se exige que ambos arrays tengan la misma longitud; si no, el `FOR` accedería a índices fuera de rango.

5. `v_i` en el `EXCEPTION`

En el bloque `EXCEPTION` se usa `v_i` y `p_producto_ids[v_i]` para reportar **qué línea** falló. `v_i` conserva su valor porque el `EXCEPTION` está en el mismo bloque que el bucle.

Alternativa con tipo compuesto (más limpia si puedes crear tipos)

Si prefieres pasar las líneas como **un solo array de pares** en lugar de dos arrays paralelos:

sql

```
CREATE TYPE tienda.linea_input AS (
    producto_id integer,
    cantidad    integer
);
```

Y el procedimiento recibiría `p_lineas tienda.linea_input[]`, recorriéndolo con:

sql

```
FOR v_i IN 1 .. array_length(p_lineas, 1) LOOP
    -- p_lineas[v_i].producto_id, p_lineas[v_i].cantidad
END LOOP;
```

Se llama así:

sql

```
CALL tienda.sp_crear_pedido_multilinea(
    1,
    ARRAY[ROW(1,2), ROW(3,1), ROW(5,4)]::tienda.linea_input[],
    NULL, NULL, NULL
```

```
);
```
Es más legible cuando hay muchos campos por línea (producto, cantidad, descuento, etc.), pero requiere crear el tipo. Los dos arrays paralelos funcionan sin tocar el esquema.

Resumen

•**Un `INSERT` en `pedidos`** fuera del bucle → todas las líneas van al mismo pedido.

•**Bucle sobre arrays paralelos** (`p_producto_ids`, `p_cantidades`) → N líneas.

•**`FOR UPDATE` por producto** → sin sobreventa concurrente.

•**`EXCEPTION` sin `COMMIT` intermedio** → atomicidad total: si una línea falla, no queda ni pedido ni stock a medias.

•**`OUT`** devuelve `pedido_id`, total y número de líneas al llamador.

ACLARACIONES:

Porque en este procedimiento no hace falta return next:

Buena pregunta — y la respuesta es corta: **`RETURN NEXT` es exclusivo de funciones (`FUNCTION`), no de procedimientos (`PROCEDURE`)**. Este código es un `CREATE PROCEDURE`, así que `RETURN NEXT` no aplica en absoluto.

1. `RETURN NEXT` solo existe en funciones

`RETURN NEXT` es una instrucción de PL/pgSQL para **devolver filas de un resultset** en funciones declaradas como `RETURNS SETOF ...` o `RETURNS TABLE (...)`. Su propósito es construir un conjunto de filas que la función devuelve al llamador.

En un `PROCEDURE` **no hay valor de retorno tipo resultset**. Un procedimiento no "devuelve filas"; ejecuta acciones y, como mucho, comunica resultados a través de **parámetros `OUT`/`INOUT`**. Por eso `RETURN NEXT` ni siquiera es válido ahí (daría error de compilación si lo pusieras).

2. Aquí los resultados se devuelven por parámetros `OUT`

Mira la firma:

sql

```
CREATE OR REPLACE PROCEDURE tienda.sp_crear_pedido_multilinea(
    IN  p_cliente_id    integer,
    IN  p_producto_ids  integer[],
    IN  p_cantidades    integer[],
    OUT v_pedido_id     integer,
    OUT v_total_pedido  numeric(12,2),
    OUT v_num_lineas    integer
```

)
Los tres valores que "devuelve" el procedimiento (`v_pedido_id`, `v_total_pedido`, `v_num_lineas`) son **parámetros `OUT`**. Dentro del cuerpo simplemente se les **asigna**:

sql

```
RETURNING pedido_id INTO v_pedido_id;   -- asigna OUT
v_total_pedido := 0;                     -- asigna OUT
v_num_lineas   := 0;                     -- asigna OUT
...
v_total_pedido := v_total_pedido + v_total_linea;  -- actualiza OUT
v_num_lineas   := v_num_lineas + 1;                -- actualiza OUT
```

Cuando el procedimiento termina, PostgreSQL devuelve esos `OUT` como una **única fila** al llamador. No hay un "conjunto de filas" que construir, así que `RETURN NEXT` no pinta nada.

Se invoca así:

sql

```
CALL tienda.sp_crear_pedido_multilinea(
    1, ARRAY[10,20], ARRAY[2,3],
    NULL, NULL, NULL   -- los OUT se pasan como NULL y se rellenan al volver
);
```

3. Comparación con tu función anterior

	Función `fn_pedidos_total`	**Procedimiento `sp_crear_pedido_multilinea`**
Tipo	`FUNCTION ... RETURNS TABLE`	`PROCEDURE`
¿Devuelve filas?	Sí, muchas (una por pedido)	No; solo valores `OUT`
Cómo devuelve	`RETURN NEXT` (o `RETURN QUERY`)	Asignando a los parámetros `OUT`
¿`RETURN NEXT`?	Necesario	No aplica

En la función, `RETURN NEXT` era el mecanismo para **emitir cada fila** del resultset. En el procedimiento no hay resultset: los "resultados" son los `OUT`, y basta con asignarlos.

4. Detalle: `RETURN` en un procedimiento

Un procedimiento **sí puede** usar `RETURN;` (sin argumentos) para **salir antes de tiempo**, pero nunca `RETURN NEXT` ni `RETURN valor`. Aquí ni siquiera se usa `RETURN;` porque el flujo cae naturalmente al final del bloque.

Resumen

- `RETURN NEXT` = instrucción de **funciones** que devuelven `SETOF`/`TABLE`.
- Este objeto es un **`PROCEDURE`**, y los procedimientos no devuelven resultsets.

- Los tres valores de salida son **parámetros `OUT`**, que se rellenan con asignaciones (`INTO` / `:=`), no con `RETURN NEXT`.
- Por eso aquí no hace falta (ni se puede usar) `RETURN NEXT`.

y si fuera exactamente igual pero una funcion. haria falta?

Sí, haría falta — pero **con matices importantes**. Si conviertes ese procedimiento en una función, tienes que decidir **qué devuelve**, y según eso usas `RETURN NEXT`, `RETURN QUERY` o simplemente `RETURN`.

Te muestro los casos.

Caso A — Función que devuelve una sola fila (los 3 valores OUT)

Si quieres mantener la misma idea (devolver `v_pedido_id`, `v_total_pedido`, `v_num_lineas` como **una fila**), lo natural es `RETURNS TABLE (...)` o `RETURNS record`, y **no necesitas `RETURN NEXT`**: basta con asignar las variables de salida y hacer `RETURN;` (o ni eso, si caes al final).

sql

```
CREATE OR REPLACE FUNCTION tienda.fn_crear_pedido_multilinea(
    p_cliente_id   integer,
    p_producto_ids integer[],
    p_cantidades   integer[]
)
RETURNS TABLE (
    v_pedido_id    integer,
    v_total_pedido numeric(12,2),
    v_num_lineas   integer
)
LANGUAGE plpgsql
AS $$
DECLARE
    v_precio_unit numeric(10,2);
    v_stock       integer;
    v_linea_id    integer;
    v_total_linea numeric(10,2);
    v_i           integer;
BEGIN
    -- ... mismas validaciones y lógica ...

    INSERT INTO tienda.pedidos (cliente_id, estado)
    VALUES (p_cliente_id, 'pendiente')
    RETURNING pedido_id INTO v_pedido_id;   -- asigna la variable de salida

    v_total_pedido := 0;
    v_num_lineas   := 0;

    FOR v_i IN 1 .. array_length(p_producto_ids, 1) LOOP
        -- ... misma lógica ...
        v_total_pedido := v_total_pedido + v_total_linea;
        v_num_lineas   := v_num_lineas + 1;
    END LOOP;
```

```
    RETURN;   -- una sola fila con los valores ya asignados a las OUT
END;
$$;
```

Aquí:

- **No** hace falta `RETURN NEXT`, porque solo emites **una fila**.
- Basta con asignar `v_pedido_id`, `v_total_pedido`, `v_num_lineas` (que son las variables de salida de `RETURNS TABLE`).
- `RETURN;` final cierra la función y devuelve esa fila.

Ojo: si pusieras `RETURN NEXT` aquí, **emitirías una fila por cada iteración del bucle**, lo cual sería incorrecto (repetirías filas).

Caso B — Función que devuelve muchas filas (una por línea de pedido)

Si en vez de devolver un resumen quisieras devolver el detalle de cada línea insertada, **ahí sí** usarías `RETURN NEXT` (o `RETURN QUERY`):

sql

```
RETURNS TABLE (
    v_pedido_id    integer,
    v_linea_id     integer,
    v_producto_id  integer,
    v_cantidad     integer,
    v_total_linea  numeric
)
...
FOR v_i IN 1 .. array_length(p_producto_ids, 1) LOOP
    ...
    INSERT INTO tienda.lineas_pedido (...)
    VALUES (...) RETURNING linea_id INTO v_linea_id;

    v_pedido_id   := ...;   -- variables de salida
    v_producto_id := p_producto_ids[v_i];
    v_cantidad    := p_cantidades[v_i];
    v_total_linea := ...;

    RETURN NEXT;   --    aquí sí, una fila por línea
END LOOP;

RETURN;
```

Aquí `RETURN NEXT` es **necesario** porque emites **N filas**.

Caso C — La forma más limpia: `RETURN QUERY`

Si no necesitas lógica procedural por fila (o solo al final), puedes hacer `RETURN QUERY SELECT ...` y **no usar ni `RETURN NEXT` ni variables OUT**:

sql

```
RETURN QUERY
    SELECT p.pedido_id, p.cliente_id, ...
    FROM tienda.pedidos p
    JOIN tienda.lineas_pedido lp ON lp.pedido_id = p.pedido_id
    WHERE p.pedido_id = v_pedido_id;
```

Regla para decidir

¿Qué devuelve la función?	¿Necesita `RETURN NEXT`?	Alternativa
Una sola fila (resumen)	No	Asignar OUT + `RETURN;`
Varias filas (detalle)	Sí (una por fila)	`RETURN QUERY`
Nada / solo efectos	No	No uses función; usa `PROCEDURE`

Resumen

- **`RETURN NEXT` no depende de "ser función" sin más**, sino de **cuántas filas devuelve**.
- Si la función devuelve **una fila** (como el procedimiento original, que solo tenía 3 OUT), **no hace falta** `RETURN NEXT`: asignas las OUT y haces `RETURN;`.
- Si la función devuelve **muchas filas** (p. ej. el detalle de las líneas), **sí hace falta** `RETURN NEXT` (o `RETURN QUERY`).
- Y si no devuelve resultset, lo correcto es que siga siendo un **`PROCEDURE`**, no una función.

pero si returns table?

Encontrados 12 resultados web

Sí, si usas `RETURNS TABLE`, la situación cambia por completo. **En la mayoría de los casos no necesitas `RETURN NEXT`, sino `RETURN QUERY`**, y si realmente usas `RETURN NEXT`, debe ir acompañado de la asignación de variables de salida.

La razón clave es que `RETURNS TABLE(...)` en PL/pgSQL es esencialmente equivalente a declarar **parámetros `OUT`** y especificar `RETURNS SETOF`. Esto significa que los nombres de columna que escribes en `TABLE(...)` se convierten directamente en "variables de salida" dentro del cuerpo de la función.

Dos formas de escritura y su lógica

1. Usar `RETURN QUERY` (recomendado, más conciso)

Esta es la forma más utilizada y clara. No necesitas gestionar variables una por una, simplemente "empujas" todo el conjunto de resultados de una consulta a la función de una vez.

sql

```
CREATE OR REPLACE FUNCTION fn_ejemplo()
RETURNS TABLE (id integer, nombre text) AS $$
BEGIN
    -- Empujar directamente todas las filas del SELECT al resultado de la
función
```

```
    RETURN QUERY
    SELECT p.id, p.nombre
    FROM personas p
    WHERE p.activo = true;
END;
$$ LANGUAGE plpgsql;
```

2. Usar `RETURN NEXT` (necesitas asignar primero las variables de salida)

Si realmente quieres usar `RETURN NEXT` dentro de un bucle `FOR`, **no puedes llevar parámetros** (como `RETURN NEXT v_registro;` reportará un error `RETURN NEXT cannot have a parameter...`). Debes asignar los valores a las variables de salida (es decir, los nombres de columna definidos en `TABLE`) y luego ejecutar `RETURN NEXT;` sin parámetros.

sql

```
CREATE OR REPLACE FUNCTION fn_ejemplo()
RETURNS TABLE (id integer, nombre text) AS $$
DECLARE
    v_rec record;
BEGIN
    FOR v_rec IN SELECT p.id, p.nombre FROM personas p LOOP
        -- 1. Asignar valores a las variables de salida (los nombres son los
nombres de columna en TABLE)
        id := v_rec.id;
        nombre := v_rec.nombre;

        -- 2. Empujar la fila actual
        RETURN NEXT;
    END LOOP;

    RETURN; -- Indica el final de la función
END;
$$ LANGUAGE plpgsql;
```

Resumen

Cuando veas `RETURNS TABLE`:

- **No necesitas** considerar `RETURN NEXT` como la primera opción.
- Si solo devuelves un conjunto de resultados, **`RETURN QUERY` es la opción más sencilla**.
- Si debes usar `RETURN NEXT`, recuerda que es "**sin parámetros**", y antes debes asignar los valores a las variables de salida con los mismos nombres que las columnas en `TABLE(...)`.